# Introduction to Pandas

## Concept of Libraries

A library is a pre-written package of code developed by expert programmers. A library comes with pre-made functions that you can import in your programs and use them to perform some common tasks. You need not know the exact details of how a library works behind the scenes. You can simply utilize the already provided functionality to make your work easier.

## Pandas

Pandas is a popular Python library that is used for data analysis. Pandas makes it easier to work with large datasets. The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

With Pandas, you can organise, clean and manipulate data, making it easier for you to perform calculations, comparisons and draw meaningful information.

## Data Structures in Pandas

A data structure is a specific way of storing and organising data so that specific operations can be performed on the data.

**Pandas basically has two data structures:**

**1. Series:** It is a single sequence of values, much like a single column of a table. A series is one-dimensional.

**2. Dataframe:** It is like a multi-column table with several series of data. It is multi-dimensional. It organises information into many rows and columns.

### Series

A series is a one-dimensional data structure containing a sequence of values of any data type (int, float, string etc). All series elements are by default labeled by numeric indices, starting from zero. But we can assign labels of different types too. We can create string-based labels as well.

**Example of a series:**

| **Index** | **Value** |
|---|---|
| 0 | Aarav |
| 1 | Bhavya |
| 2 | Chiku |
| 3 | Deep |

To access “Bhavya”, we would write SeriesName[1].

**Example of a series with a different index:**

| **Index** | **Value** |
|---|---|
| France | Paris |
| Britain | London |
| India | Delhi |
| USA | WashingtonDC |

Here, the names of countries are acting like an index to access the names of the corresponding capital cities.

To access “Paris”, we would write SeriesName[‘France’].

### *Different ways of creating Series*

1. **Creation of Series from a List of values:**
   series1 = pd.Series([10, 20, 30])
   In this case, the values will automatically assigned numeric indices, starting from zero. The value 10 will have an index label of 0, 20 will be at index 1 and so on.

   Example:

   | Index | Value |
   |---|---|
   | 0 | 10 |
   | 1 | 20 |
   | 2 | 30 |

2. **Creation of Series from a Dictionary:**
   series1 = pd.Series( {'India': 'Delhi', 'UK': 'London', 'France': 'Paris'} )
   In this case, the keys of the dictionary (i.e. India, UK, France become the indices of the series and the corresponding values become the elements of the series.

   Example:

   | Index | Value |
   |---|---|
   | India | Delhi |
   | UK | London |
   | France | Paris |

3. **Creation of Series from NumPy Arrays:**
   import numpy as np
   import pandas as pd
   array1 = np.array([10, 20, 30])
   series1 = pd.Series(array1)

### *Attributes of Series*

Attributes are certain properties of Pandas series that we can access or change.

**Some common attributes are:**

1. **name** : used to assign a name to the series.
   E.g. s1.name = 'Mark List'
2. **index.name** : assigns a name to the index of the series.

   E.g. s1.index.name = 'Students'
3. **values** : prints a list of the values in the series.
4. **size** : prints the number of values in the series.
5. **empty** : prints True if the series is empty and False otherwise.

### *Common Methods of Series*

Some common methods that can be used with series are:

1. head(n): Returns the first n number of elements from the top of a series. If n is not specified, then it returns first 5 elements by default. E.g. S.head() or S.head(10)
2. tail(n): Returns the last n number of elements from the bottom of a series. If n is not specified, then it returns last 5 elements by default. E.g. S.tail() or S.tail(10)
3. count(): Used to count the number of non-NaN values in a series. E.g. S.count()

## *Mathematical operations on Series*

**Addition:**

series1 = pd.Series([10, 20, 30])

series2 = pd.Series([10, 20])

series3 = series1 + series2

series3 will be [20, 40, NaN]

Or,

Series3 = series1.add(series2, fill_value = 0)

**(Here, fill_value = 0 specifies that if there is no matching element in any of the series, it will be by default considered as 0)**

Then, series3 will be [20, 40, 30], **because it has "added" 0 as the third element in series2 before performing the addition.**

Subtraction, multiplication, and division can be performed in the same way using either the mathematical operators (-, * or /), or by using methods: sub(), mul() and div().

## DataFrame

A DataFrame is atwo-dimensional data structure in Python. It is like a complete table with several columns of data.

Example:

| R. No | Name | Age | Marks |
|---|---|---|---|
| 1 | Aman | 13 | 67 |
| 2 | Ankit | 12 | 89 |

## *Difference between series and DataFrame:*

1. Series: One-dimensional array-like structure capable of holding any data type.

   DataFrame: Two-dimensional labeled data structure with columns of potentially different types.

2. Series: Can be thought of as a single column of a DataFrame.

   DataFrame: Consists of multiple Series arranged in a tabular fashion.

3. Series: Indexed by a single set of labels.

   DataFrame: Indexed by both row and column labels.

4. Series: Can perform vectorized operations.

   DataFrame: Allows for operations across rows and columns simultaneously.

5. Series: Typically used for one-dimensional data analysis.

   DataFrame: Used for two-dimensional data analysis and manipulation.

## CSV Files

CSV stands for "Comma-Separated Values." It's a simple and widely used file format for storing tabular data, such as spreadsheets or databases. CSV files contain rows of data, where each row consists of one or more fields separated by commas (or other delimiters like semicolons or tabs).

A CSV can be easily created in any text editor like Notepad or a spreadsheet program like Excel. Moreover, CSV files can also be generated "programmatically" by writing codes in a programming language like Python.

CSV files are useful for several reasons:

1. **Data Interchange**: CSV files are a common way to exchange data between different applications, as many software programs and programming languages can read and write CSV format.

2. **Simplicity**: CSV is a plain text format, making it human-readable and easy to create and edit with basic text editors like Notepad.

3. **Compatibility**: It's a format that's not tied to any specific software, making it platform-independent and versatile.

4. **Small File Size**: CSV files tend to have smaller file sizes compared to binary formats, which can be beneficial for storage and data transmission.

5. **Integration**: Many data analysis and visualization tools, like Excel, Python's Pandas library, and various database systems, support CSV import/export, making it a convenient choice for data manipulation.

6. **Automation**: CSV files can be generated and processed "programmatically", making them ideal for automating data-related tasks.

### *Reading and writing CSV files using Pandas*

The Pandas library allows us to easily import or export data from/to CSV files.

We can read as well as write CSV files using two Pandas functions:

**1. pd.read_csv('filename and path')**

**2. df.to_csv('filename and path')**

We can customise the way CSV files are read or written by supplying different parameters to these functions inside the brackets.

## Practice Questions based on Series and DataFrame:

**Practice Questions based on Pandas Series Creation**

1. Create a Pandas series to hold the names of all months indexed by the month numbers, starting from 1 (1: January, 2: February and so on)

2. Create a Pandas series of any five fruits. Slice the series to print the names of last two fruits.

3. Create a Pandas series using a dictionary, where keys of the dictionary are the names of any 3 batsmen and values are the respective runs scored by them in the last match.

4. Create a Pandas series that contains the sales amount of the last 3 years, indexed by their respective year numbers.

5. Imagine that you are provided with a Pandas series containing 5,000 values. How will you display just the last five values in the series?

6. Create a Pandas series containing all even numbers between 1 and 1000.

**Practice Questions based on Pandas DataFrame (Creation and Basic Operations):**

1. How many rows and columns will the following DataFrame have?

df = pd.DataFrame({ 'Name': ['Robin', 'Aryan', 'Isha'], 'Marks': [78, 90, 83] })

2. Create a DataFrame from the following 2 Pandas series, containing marks of Student1 and Student2 in last 5 tests:

S1 = pd.Series([8, 9, 9, 6, 10])

S2 = pd.Series([6, 5, 7, 8, 10])

3. Write a Python code to delete a row labelled as "R4" from a DataFrame.

4. Create a DataFrame as shown below:

| | Eng | Hindi |
|---|---|---|
| Tarun | 14 | 15 |
| Reena | 12 | 16 |

a) Write a code to display only rows where marks in Hindi > 15

b) Add one more column "Maths" and insert 15 marks for both the students.

c) Add one more student "Rishabh" with 12 marks in all the subjects.

d) Rename "Eng" as "English"

e) Drop the column "Hindi"

5. What is the use of 'axis' argument?

6. A teacher wants to know the marks scored by a student with roll number 4. Assuming that we have a DataFrame containing two columns: 'Name' and 'Marks', and all rows indexed by the respective roll numbers of students, how will you help the teacher?

7. Write a Python code to access a value located in row 6 and column 3.

8. How will you display the last 5 rows of a given DataFrame named "df"?

9. Differentiate between loc() and iloc().

10. Create a sample DataFrame from a list of dictionaries, containing at least 2 rows and 2 columns.